

엣지 배포를 위한 ViT·CNN 포렌식 백엔드 모델 압축 기법 최신 동향

배경과 배포 제약

Raspberry Pi 5는 2.4GHz 쿼드코어 Arm Cortex-A76(BCM2712) 기반이며, LPDDR4X-4267 메모리(8GB SKU 포함), PCIe 2.0 x1 등으로 확장 가속기 연결이 가능하다는 점이 엣지 추론 최적화의 출발점입니다. ¹

옵션 가속기인 Hailo-8L 계열은 최대 13 TOPS, “typical power” 약 1.5W를 표방하며, ARM 호스트와 PCIe 연결을 전제로 하고, TensorFlow/TFLite/Keras/ PyTorch/ONNX 등 프레임워크 생태계와의 통합(컴파일러·툴체인)을 강조합니다. ²

또한 Raspberry Pi 생태계 관점에서, (구형) AI Kit는 “더 이상 생산하지 않으며” AI HAT(+) 계열을 권장한다는 점이 공급/부품 선정의 현실적 제약입니다. ³

이 연구의 최종 목표(총 <500MB, 총 <1s)는 “가중치 저장 크기”뿐 아니라 (1) 런타임(연산자 커널/스케줄링), (2) 메모리 피크(활성값·중간 버퍼), (3) 멀티모델 오케스트레이션(동시 로딩/캐시/온디맨드)에 의해 좌우됩니다. 특히 ViT/CLIP 계열은 LayerNorm·GELU·Softmax·어텐션·토큰 처리 등에서 런타임 특성이 CNN과 다르므로, **아키텍처 교체(경량 백본) + 정수 양자화(QAT/PTQ) + 구조적 프루닝**을 “모델별로” 조합해 Pareto를 만드는 접근이 일반적입니다. ⁴

비전 트랜스포머 압축

CLIP·ViT를 “더 작은 백본으로 대체”하는 최신 경향

Pavan Kumar Anasosalu Vasu ⁵ et al., “MobileCLIP: Fast Image-Text Models through Multi-Modal Reinforced Training”, CVPR 2024.

MobileCLIP은 모바일 친화적 이미지/텍스트 인코더 쌍을 설계하고, 오프라인 “강화 데이터셋”을 통해 대형 CLIP 앙상블의 지식을 학습 데이터 형태로 주입하는 방식으로 학습 효율과 추론 효율을 동시에 노립니다. ⁶

- 압축/가속: 논문은 **MobileCLIP-S0가 OpenAI ViT-B/16 CLIP 대비 약 5× 빠르고 3× 작으면서 평균 정확도는 동급**이라고 보고합니다. ⁷

- 지표/하드웨어: iPhone12 Pro Max(iOS 17)에서 Core ML로 지연시간을 측정하며, 소형 설정에서 MobileCLIP 계열의 (img+txt) ms 단위 지연과 zero-shot ImageNet 성능을 표로 제시합니다. ⁸

- 포렌식 적용 포인트: FatFormer류(“CLIP 백본 + 얇은/부가 모듈”) 구조는 **백본 교체**가 가장 큰 압축 레버리지가 됩니다. MobileCLIP의 “latency-accuracy tradeoff”는 CLIP 백본 교체 후보로 실용성이 높습니다. ⁶

Kan Wu ⁹ et al., “TinyCLIP: CLIP Distillation via Affinity Mimicking and Weight Inheritance”, ICCV 2023.

TinyCLIP은 “교차모달 affinity(정렬 관계)”를 모사하는 증류와, 교사 가중치 일부를 학생에 상속하는 학습을 결합합니다. ¹⁰

- 압축: CLIP ViT-B/32의 **사이즈를 50% 축소하면서 zero-shot 성능을 유사하게 유지**할 수 있다고 설명합니다. ¹⁰

- 극단 경량 사례: TinyCLIP-ViT-8M/16이 ImageNet zero-shot top-1 41.1%를 달성하면서 “원본 CLIP ViT-B/16”보다도 높고, 파라미터는 8.9% 수준이라고 주장합니다. ¹⁰

- 포렌식 적용 포인트: “CLIP 백본 축소 + 성능 회복(증류)”이라는 구성 자체가 FatFormer의 백본 교체 전략과 직접 호환됩니다. ¹⁰

Chuangang Yang ¹¹ et al., “CLIP-KD: An Empirical Study of CLIP Model Distillation”, arXiv 2023(업데이트 2024).

CLIP-KD는 다양한 종류(관계/특징/그래디언트/대조) 전략을 비교하고, 단순 MSE 기반 **feature mimicry**가 강력하게 작동한다는 경험적 결론을 제시합니다. ¹²

- 정확도 보전: ViT-L/14 교사를 사용했을 때, ViT-B/16·ResNet-50 학생의 ImageNet zero-shot top-1이 각각 57.5%, 55.4%까지 올라가며, “KD 없이 학습한 CLIP” 대비 +20%p 내외 향상을 보고합니다. ¹²

- 교차 아키텍처: MobileViT-S 같은 학생에서도 zero-shot 정확도 개선을 보고하며, 교사-학생 아키텍처가 같아야 하는 제약을 완화하려는 동기가 명시됩니다. ¹²

- 포렌식 적용 포인트: “ViT/CLIP 교사(현재 FatFormer) → 더 작은 학생 백본(예: MobileCLIP, MobileViT, EfficientNet 계열)”로 **교차 아키텍처 종류**를 설계할 때 근거가 됩니다. ¹²

ViT 프루닝

Xinjian Wu ¹³ et al., “PPT: Token Pruning and Pooling for Efficient Vision Transformers”, OpenReview(제출: 2023, ICLR 2024 철회).

PPT는 레이어별로 “토큰 프루닝”과 “토큰 풀링”을 선택적으로 적용해 중복·무관 토큰을 줄이는 프레임워크를 제안합니다. ¹⁴

- 성능/효율: DeiT-S에서 **FLOPs 37%+ 절감** 및 **throughput 45%+ 개선**, “ImageNet 정확도 드롭 없음”을 주장합니다. ¹⁴

- 엣지 함의: CLIP/ViT의 입력(패치 토큰) 수에 의해 어텐션 비용이 커지는 구조에서는 토큰 축소(프루닝/머징/풀링)가 **지연시간을 직접 깎는** 경로가 됩니다. ¹⁴

Lu Yu ¹⁵ & Wei Xiang ¹⁶ , “X-Pruner: eXplainable Pruning for Vision Transformers”, CVPR 2023.

X-Pruner는 클래스 기여도(설명가능성) 기반 마스크를 학습해 attention head·행 단위 등 “구조적 단위”를 가지치기합니다. ¹⁷

- 압축/정확도: ILSVRC-12에서 DeiT-T/S 등 대상으로 **연산량(FLOPs) 51.3%~66.1% 절감**을 보고하면서, top-1 정확도 드롭은 0.72%~1.1% 범위라고 요약합니다. ¹⁸

- 포렌식 적용 포인트: “구조적 프루닝”은 실제 ARM 커널에서 가속이 나오기 쉬운 반면, 비정형 sparsity는 엣지에서 이득이 불확실합니다(커널/라이브러리 의존). X-Pruner는 구조적 단위라는 점에서 실전성이 높습니다. ¹⁹

Kaixin Xu ²⁰ et al., “LPViT: Low-Power Semi-structured Pruning for Vision Transformers”, arXiv 2024 (ECCV 포스터로도 소개).

LPViT는 블록 구조 희소성을 활용해 하드웨어 친화적 행렬곱 최적화를 목표로 하는 “반구조적(semi-structured)” 프루닝을 제안합니다. ²¹

- 효율 지표: DeiT-B에서 전용 하드웨어 3.93×, GPU 1.79× 속도 향상, 실GPU에서 1.4× 전력 감소 등을 요약적으로 보고합니다. ²²

- 포렌식 적용 포인트: 엣지에서 **전력/발열**이 중요한 경우(특히 Pi 5 케이스·팬리스 설계) “전력까지 목적함수에 포함”하는 프루닝 접근이 참고가 됩니다. ²³

ViT 양자화

Zhuguanyu Wu ²⁴ et al., “FIMA-Q: Post-Training Quantization for Vision Transformers by Fisher Information Matrix Approximation”, CVPR 2025.

FIMA-Q는 FIM 근사(DPLR-FIM)를 통해 ViT PTQ의 정확도 하락을 줄이고, “표준 균일(Uniform) 양자화”로도 강한 성능을 노립니다. ²⁵

- PTQ vs QAT: 논문은 QAT가 높은 정확도를 내지만 전체 데이터 재학습 비용이 크고, PTQ는 소량의 (무라벨) 캘리브레이션 데이터로 비용이 낮다는 맥락을 명시합니다. ²⁶

- 4bit 예시(참지/세그): COCO에서 Swin-T/S 기반 Mask R-CNN·Cascade Mask R-CNN의 W4/A4 양자화 결과를 표로 제시하며, FIMA-Q가 동일 설정에서 경쟁 방법 대비 우수한 AP를 보고합니다. ²⁷

- 포렌식 적용 포인트: ViT 기반 백본(특히 CLIP-ViT)을 INT8 이하로 내릴 때, “그냥 PTQ”는 손실이 클 수 있고, 블록 재구성·2차 정보 기반 손실 등 **ViT-특화 PTQ**를 참고해야 함을 시사합니다. 25

포렌식 모델 지식 종류

FatFormer 계열에서의 “백본 교체 + 어댑터 재학습” 질문

FatFormer는 “사전학습 CLIP 비전-언어 공간” 위에서 **Forgery-Aware Adapter(FAA)** 로 이미지·주파수 단서(예: DWT)를 통합해 위조 표현을 적응시키고, **Language-Guided Alignment(LGA)** 로 텍스트 프롬프트와의 대조 목적을 통해 일반화를 강화한다는 설계를 명시합니다. 28

또한 4-class ProGAN으로 튜닝한 후 “unseen GANs 평균 98% 정확도”, “unseen diffusion 95% 정확도”를 보고하며, CLIP-ViT-L/14 백본 사용 방법(체크포인트 다운로드)을 README에 기재합니다. 28

핵심 질문: “ViT-L/14를 더 작은 CLIP로 바꾸면 어댑터만 재학습 가능?”

- 원리적으로는 “가능한 쪽”에 가깝습니다. FatFormer는 애초에 “고정된 CLIP 위에 적응 모듈을 붙여 위조 표현 학습이 부족해지는 문제를 해결한다”는 문제정의를 갖고, FAA/LGA를 통해 “forgery adaptation”을 수행한다고 설명합니다. 즉, **백본을 freeze한 채 부가 모듈만 학습**하는 패턴 자체를 채택합니다. 28

- 다만 “그대로 어댑터만 갈아끼우기”는 백본 구조/차원에 강하게 의존합니다. ViT-L/14 ↔ ViT-B/16은 히든 차원·블록 수·토큰화 규칙 등이 달라 어댑터 파라미터 shape가 바뀌는 것이 보통이며, MobileCLIP처럼 인코더 블록 설계가 크게 다르면 삽입 위치/특징 맵의 의미가 달라집니다. 따라서 현실적으로는 ‘**새 백본에 맞게 어댑터를 포팅(차원 정렬 포함)한 뒤, 어댑터(+최종 헤드) 재학습**’이 필요합니다. 29

- 성능 회복 수단으로는 **교사-학생 종류**가 매우 유효합니다. CLIP-KD는 ViT-L/14 교사를 사용해 ViT-B/16·ResNet-50 학생의 zero-shot 성능을 크게 끌어올릴 수 있음을 보이며, 아키텍처 동일성 제약을 낮추는 방향을 강조합니다. 12

- 결론적으로, “어댑터만 학습”은 **(A) 동일 계열 ViT로 다운스케일할 때가 가장 쉽고, (B) MobileCLIP/TinyCLIP 같은 대폭 변경 백본**은 어댑터 포팅 + (가능하면) distillation + 부분 미세조정(예: 마지막 몇 블록)까지 포함한 설계가 안전합니다. 30

포렌식 분야에서 “다중 모델/다중 단서”를 단일 학생으로 압축하는 예

Zeqin Yu 31 et al., “Reinforced Multi-teacher Knowledge Distillation for Efficient General Image Forgery Detection and Localization”, arXiv 2025.

이 작업은 copy-move/splicing/inpainting 등 위조 유형별 “전문화된 다중 교사”를 두고, 강화학습 기반 teacher selection으로 단일 학생(Cue-Net)에 지식을 전달하여 “다중 단서 양상블을 단일 모델로 흡수”하는 형태를 취합니다. 32

- “양상블→단일화” 압축: 추론 시 여러 모델을 돌리는 대신 학생 1개로 통합하는 방향이며(모델 수·운영단 복잡도 감소), 논문은 다양한 데이터에서 평균 AUC 개선을 보고합니다. 32

- 효율 지표(상대 비교): 보조 자료 표에서 “Ours 38.2 ms/img, 메모리 3147MB, 평균 AUC 0.903” 및 MVSS-Net/TruFor 등과의 속도 비교를 제시합니다(측정 환경은 논문 설정을 따라야 하며, 옛지 CPU 값으로 해석하면 안 됩니다). 33

- 포렌식 적용 포인트: 귀 시스템처럼 멀티 에이전트/멀티 백엔드가 있는 경우, **교사(기존 4개)→학생(1~2개)** 구조로 distill하여 “모델 수 자체”를 줄이는 것이 총 지연시간·메모리·운영 복잡도를 한 번에 줄이는 강력한 옵션입니다. 33

“미세한 신호(스태가/노이즈)” 영역에서 프루닝이 성립하는 사례

Eunseong Hong 34 et al., “Lightweight image steganalysis with block-wise pruning”, 2023.

스태가 분석은 포렌식과 마찬가지로 “아주 미세한 통계적/주파수 신호”에 민감한 영역인데, 이 연구는 EfficientNet-B0에서 블록을 점진 제거하는 전략으로 경량화를 시도합니다. 35

- 압축: EfficientNet-B0 변형을 **9.58× 더 작게**(파라미터) 만들고 FLOPs도 2.16× 감소시키면서, BOSSBase와 ALASKA#2에서 baseline과 동급 수준의 정확도를 보고합니다. 35

- 포렌식 적용 포인트: “경량화가 곧 신호 손실”로 직결될 것이라는 우려에 대해, **도메인 맞춤 구조 제거(블록 단위) + 평가 설계**로 실용적 타협점이 가능함을 보여줍니다. 35

포렌식 모델 양자화

포렌식 모델이 양자화에 더 민감할 수 있는 이유와 근거

AI-generated/딥페이크 검출은 미세한 텍스트·주파수 단서를 잡아내야 하므로, “단순 저정밀화”가 디테일 손실을 유발할 수 있다는 문제의식이 최근 논문에서 직접적으로 제기됩니다. 36

특히 IJCAI 2025의 딥페이크 검출 양자화 연구는, 저비트 양자화에서 “세부 손실(rounding에 의한 디테일 손실)”을 완화하기 위해 **full-precision shortcut**과 **활성 분포 재분배(Shifted Logarithmic Redistribution)** 같은 설계를 넣어야 한다고 설명합니다. 이는 포렌식 계열이 양자화에 “무조건 강인하지 않다”는 실무적 신호로 해석 가능합니다. 37

포렌식/딥페이크 검출에서의 저비트 양자화 성공 사례

Renshuai Tao 38 et al., “Unlocking the Potential of Lightweight Quantized Models for Deepfake Detection”, IJCAI 2025.

이 논문은 딥페이크 검출을 대상으로 저비트 양자화 프레임워크(CQB, SLR quantizer)를 제안합니다. 37

- 압축/효율: 연산량 **10.8× 감소**, 저장 요구 **12.4× 감소**를 주장하면서도 성능 유지를 보고합니다. 37

- 실무 시사점: (1) 단순 QAT/PTQ만이 아니라, (2) 아키텍처를 “양자화 친화적으로” 바꾸는 것이 정확도 보전에 중요하다는 메시지입니다. 포렌식 CNN(MVSS-Net, CAT-Net 계열)도 residual/shortcut, 입력 표현(노이즈 residual 등) 관점에서 유사한 전략을 적용할 여지가 있습니다. 39

PTQ vs QAT, 그리고 틀체인 관점

- ViT 계열은 “ViT-특화 PTQ”가 활발히 연구되며, FIMA-Q처럼 2차 정보 근사와 블록 재구성으로 저비트 정확도 하락을 줄이려는 방법이 대표적입니다. 25
- ONNX Runtime은 모델 최적화/양자화 기능을 문서화하고 있으며, GPU에서는 TensorRT EP를 활용한 절차를 별도로 안내합니다(단, Pi 5 CPU 환경과는 다름). 40
- 엣지에서의 실무 패턴은 (1) CPU에서 검증 가능한 INT8(QAT 또는 보수적 PTQ) → (2) 커널/EP(XNNPACK/ACL 등)로 실행 최적화 → (3) 필요 시 mixed-precision(일부 블록 FP16/FP32 유지)으로 복구하는 흐름이 일반적입니다. ONNX Runtime은 XNNPACK EP가 “Arm 기반에서 최적화된 FP 연산자” 집합임을 명시합니다. 41

구조적 프루닝 및 멀티스트림 아키텍처

듀얼 브랜치(CAT-Net DCT+RGB)에서 “한 스트림 통째 제거” 가능성

CAT-Net류는 RGB 공간 특징과 DCT/압축 단서를 병렬로 활용하는 계열인데, “스트림 통째 제거”는 **정확도-일반화 손실 위험이 커서 연구 근거만으로 단정하기 어렵고**, 보통은 아래 순서가 안전합니다.

1) 스트림별 ablation(단일 스트림 성능/강건성) → 2) 스트림 간 지식 증류(두 스트림 교사 → 단일 학생) → 3) 남긴 스트림에 구조적 프루닝/양자화 적용. 이 “멀티→싱글 distill” 전략이 포렌식에서 효율화에 쓰일 수 있다는 맥락은 Re-MTKD류에서 확인됩니다. 42

또한 2026년 MNVFusion 계열 연구는 “RGB만 과신하지 말라”는 문제의식으로 noise-view 기반 융합을 강조해, 스트림 제거가 오히려 일반화 성능을 해칠 수 있음을 시사합니다(직접적으로 ‘CAT-Net 스트림 제거’를 다루진 않음). 43

구조적 프루닝의 일반 원칙과 포렌식 적용

구조적 프루닝은 채널/필터/블록처럼 연산 그래프에서 “연속된 구조”를 제거해 실제 하드웨어 가속으로 연결시키는 접근이며, 최근 서베이에서도 구조적 프루닝을 체계화해 정리합니다. ⁴⁴

- 필터 중요도 기준(L1-norm, Taylor 등)은 폭넓게 쓰이지만, 포렌식은 신호가 미세하여 “정확도 민감도”가 더 클 수 있으므로, **포렌식 데이터(압축·리사이즈·노이즈)로 민감도 분석**을 병행해야 합니다. ⁴⁵

- 전이 학습 상황에서는 “프루닝 + 어댑터(PEFT)” 결합이 실용적일 수 있습니다. Structured Pruning Adapters는 프루닝과 어댑터를 결합해, 단순 프루닝+파인튜닝 대비 파라미터 효율(10× fewer params)을 주장하며, 높은 프루닝 비율에서 정확도 장점을 강조합니다. ⁴⁶

- FatFormer처럼 “백본은 크게 건드리지 않고, 소량 모듈로 적응”하는 구조는 SPAs류 아이디어와 궁합이 좋습니다(단, 실제 적용은 구현/평가 필요). ⁴⁷

Raspberry Pi 5와 Hailo 기반 최적화

런타임 선택: PyTorch Mobile, ExecuTorch, ONNX Runtime, TFLite

PyTorch Mobile은 커뮤니티/포럼 레벨에서 “deprecated”로 언급되며, PyTorch 측은 ExecuTorch를 온디바이스 런타임으로 적극 제시(베타 선언, 모바일/엣지 지원)합니다. ⁴⁸

ONNX Runtime은 모바일/엣지 가이드를 통해, 비양자화 모델은 XNNPACK, 양자화 모델은 CPU EP부터 시작하는 식의 실무 지침을 제공합니다. ⁴⁹

또한 Arm Compute Library(ACL) EP는 Arm CPU 가속 옵션으로 문서화되어 있습니다. ⁵⁰

Raspberry Pi 5에서의 “CPU-only” 추론 기준선

공개된 실험 중 하나는 Raspberry Pi 5에서 ONNX Runtime으로 여러 경량 모델의 **ms/img, FPS, Peak RAM, CPU 사용률**을 1-thread/4-thread로 보고합니다(작업 도메인은 농업 분류이지만, “Pi 5 + ORT”의 추론 기준선으로 참고 가능). ⁵¹

- 예: MobileNetV3-Small은 1-thread에서 19.71ms/img(≈50.7 FPS), 4-thread에서 9.00ms/img(≈111 FPS)로 보고되고, Peak RAM은 약 99MB 수준으로 제시됩니다. ⁵¹

- 하이브리드(모바일 ViT 계열) 예: MobilePlantViT는 1-thread 252.26ms/img(≈3.96 FPS), 4-thread 101.46ms/img(≈9.86 FPS), Peak RAM 약 130MB로 보고됩니다. ⁵¹

위 값은 귀 포렌식 모델과 네트워크 구조가 다르므로 “절대값”으로 옮기면 안 되지만, **Pi 5 CPU에서 (i) 경량 CNN은 수십~수백 FPS도 가능, (ii) 하이브리드 ViT는 10 FPS 내외까지 떨어질 수 있음**이라는 방향성은 런타임/아키텍처 선택에 직접 영향을 줍니다. ⁵²

Hailo-8L 및 컴파일 파이프라인

Raspberry Pi Ltd ⁵³ 는 AI Kit 업데이트에서 Hailo Dataflow Compiler(DFC)가 “자체 데이터/모델을 컴파일해 AI Kit에서 실행” 가능하도록 제공되었다고 설명합니다. ⁵⁴

Hailo ⁵⁵ 측 GitHub 예제는 DFC가 Hailo-8/8L 타깃으로 네트워크를 컴파일하는 도구이며, 개발자 존에서 내려받는 구조임을 명시합니다. ⁵⁶

Hailo 제품 브리프는 ARM 호스트/PCIe 연결, 13 TOPS, typical 1.5W, 그리고 TensorFlow/TFLite/Keras/PyTorch/ONNX 생태계 지원을 표기합니다. ²

또한 Raspberry Pi 문서는 AI HAT+ 라인업과 AI Kit의 기능적 동등성(동일 사양의 Hailo-8L 변형)을 설명합니다. ⁵⁷

Hailo를 실제로 쓰려면, (1) 모델을 ONNX/TFLite 등으로 정리한 뒤 (2) DFC로 HEF로 컴파일하고 (3) 런타임에서 파이프라인을 구성하는 흐름이 일반적입니다. ⁵⁸

다만 포렌식 모델은 비표준 연산(주파수 변환, 특수 후처리, 커스텀 레이어)이 포함될 가능성이 높아, “컴파일 가능한 연산 서브그래프를 최대화하도록” 모델을 재구성해야 Hailo의 TOPS를 실제 지연시간 감소로 연결할 수 있습니다(이 부

분은 DFC가 ‘고급 사용자’용으로 제공되며, 커스텀 모델 컴파일을 전제로 한다는 Raspberry Pi 측 설명과 일관됩니다).

59

드롭인 경량 대안과 모델별 압축 요약

드롭인 대안의 “근거 있는 후보군”

FatFormer(현재 CLIP ViT-L/14 기반) 대체 후보

- **MobileCLIP-S0/S1/S2 계열**: “모바일 지연시간-정확도 tradeoff”를 목표로 설계되어, CLIP 백본 교체 후보로 강력합니다. ⁶
- **TinyCLIP 계열(예: ViT-8M/16)**: 극단적 파라미터 절감(8.9% 파라미터)과 zero-shot 성능 유지/개선을 주장해, “<500MB 총량” 목표에서 매우 공격적인 옵션입니다. ¹⁰
- **EfficientViT 백본**: CNN-ViT 하이브리드/메모리 효율 설계를 통해 MobileNetV3·MobileViT 대비 throughput 향상을 보고하며, ONNX 변환 시 속도 이득을 강조합니다. ⁶⁰
- **증류로 성능 회복**: 교사(기존 FatFormer/CLIP-L/14) → 학생(위 후보)로 CLIP-KD를 적용하면 작은 백본에서 zero-shot 성능 격차를 줄일 수 있음을 경험적으로 보여줍니다. ¹²

CAT-Net v2 / MVSS-Net / Mesorch(CNN 계열) 대체 후보

- **양자화-친화적 설계 참고**: “디테일 손실을 full-precision shortcut으로 보완”하는 딥페이크 양자화 연구는, 포렌식 CNN에도 residual/skip을 활용해 INT8 정확도 하락을 줄일 수 있다는 설계 힌트를 제공합니다. ⁶¹
- **블록 단위 제거(포렌식 유사 도메인)**: 스테가 분석에서 EfficientNet-B0를 9.58× 줄이면서 정확도를 유지한 결과는, MVSS-Net/메소르크류에서도 “블록 제거/경량 백본 교체”가 성립할 수 있음을 시사합니다. ³⁵
- **멀티스트림 → 단일 학생(증류)**: CAT-Net처럼 듀얼 스트림이 무거운 경우, Re-MTKD류처럼 다중 단서를 하나의 학생 모델로 distill하는 접근이 “모델 수·스트림 수”를 동시에 줄일 수 있습니다. ⁴²

모델별 “압축 방법 → 예상 크기 절감 → 정확도 영향” 요약표

아래 표의 “예상 크기 절감”은 **가중치 저장 관점(이론적 배수)**을 우선 적용하고, 포렌식 특성상 성능 하락 위험을 별도 표기했습니다. INT8은 원칙적으로 FP32 대비 4×, FP16 대비 2×의 저장 절감이 기본입니다(단, 런타임이 해당 정수 커널을 제대로 쓰는지에 따라 “속도”는 달라집니다). ⁶²

대상 모델	압축 기법(권장 우선순위)	예상 저장 크기 절감	정확도 영향(리스크 요약)	근거/유사 근거
FatFormer (CLIP ViT-L/14 + FAA/DWT)	백본 교체 (MobileCLIP / TinyCLIP / EfficientViT) + (필요 시) 교사-학생 증류	“백본 자체”가 가장 큰 절감 레버 (사례: MobileCLIP이 CLIP-B/16 대비 3× 작다고 보고) ⁷	중간~높음 (포렌식 분포 적용 필요)	MobileCLIP·TinyCLIP·EfficientViT가 “모바일 효율 개선”을 직접 목표로 보고 ⁶³ , CLIP-KD가 소형 학생 성능 회복을 보고 ¹²

대상 모델	압축 기법(권장 우선순위)	예상 저장 크기 절감	정확도 영향(리스크 요약)	근거/유사 근거
	토큰 프루닝/풀링 (PPT류) (가능 시)	FLOPs 37%+ 감소(특정 ViT에서) ¹⁴	낮음~중간 (정확도 드롭 없다고 주장한 케이스 존재)	PPT가 ImageNet에서 “무정확도 저하” 주장 ¹⁴
	ViT PTQ(예: FIMA-Q류) 또는 QAT	INT8(4×), 4bit(가중치 기준 8×) 가능성	중간 (ViT는 저비트에서 급락 가능, ViT-특화 PTQ 필요)	FIMA-Q가 저비트에서 정확도 개선 보고 ²⁵
CAT-Net v2 (DCT+RGB 듀얼)	INT8(QAT 권장) + 구조적 채널/블록 프루닝	INT8만으로 ~4×	중간 (압축 단서 손실 가능)	스테가 분석에서 블록 제거로 정확도 유지 사례 ³⁵ ; 구조적 프루닝 일반 정리 ⁴⁴
	듀얼→싱글(종류/단일화)	“스트림 수” 축소로 구조 자체 경량화	중간~높음(일반화 손실 가능)	멀티 교사→단일 학생으로 효율/성능 균형 사례 ⁴²
MVSS-Net (노이즈-멀티스케일)	INT8(QAT) + 블록/스케일 축소 (도메인 맞춤 프루닝)	INT8 ~4× + 구조 축소	중간 (노이즈 단서가 저정밀에 민감할 수 있음)	저비트에서 디테일 손실을 보완하는 설계 필요성이 지적됨 ⁶¹
Mesorch (공간 로컬라이제이션)	경량 세그 백본으로 교체 + 종류 + INT8	백본 교체 + INT8(~4×)	중간 (마스크 품질 저하 가능)	Re-MTKD가 탐지+로컬라이제이션을 단일 모델로 효율화(속도 비교 포함) ³³

“<500MB / <1s” 목표에 대한 실전적 조합 제안

- **CNN 3종(CAT-Net, MVSS-Net, Mesorch):** INT8(QAT 우선)만으로도 저장 크기는 이론상 4× 절감이 가능해(각각 대략 150→37.5MB, 120→30MB, 100→25MB 수준) 총량을 크게 줄일 수 있습니다. 다만 포렌식은 디테일 민감도가 있을 수 있으므로, IJCAI 2025처럼 shortcut/입력표현 등을 양자화 친화적으로 설계하는 방향이 필요할 수 있습니다. ⁶⁴
- **FatFormer:** 총량/지연시간의 병목이므로, “백본 교체 + (필요 시) 증류”가 1순위입니다. MobileCLIP이 보여주는 모바일 지연시간 지표와 “3× smaller” 주장, TinyCLIP의 극단 축소 결과는 ‘교체 후보’의 강한 근거입니다. ⁶⁵
- **멀티 모델 오케스트레이션:** Re-MTKD류처럼 “여러 단서/모델을 단일 학생으로 distill”하는 방향은, 귀 시스템의 4백엔드를 2개 이하로 줄이는 장기 해법이 될 수 있습니다. ³³
- **런타임:** Pi 5에서는 ONNX Runtime + (XNNPACK/ACL 등)로 “가장 먼저 CPU-only INT8/FP16”을 안정화하고, Hailo는 컴파일 가능한 서브그래프를 확실히 만들 수 있을 때만 투입하는 편이 리스크가 낮습니다. ⁶⁶

¹ <https://pip.raspberrypi.com/documents/RP-008348-DS-raspberry-pi-5-product-brief.pdf>

<https://pip.raspberrypi.com/documents/RP-008348-DS-raspberry-pi-5-product-brief.pdf>

² ³¹ <https://hailo.ai/wp-content/uploads/2023/10/Hailo-8L-Product-Brief-Rev1.1.pdf>

<https://hailo.ai/wp-content/uploads/2023/10/Hailo-8L-Product-Brief-Rev1.1.pdf>

³ <https://www.raspberrypi.com/documentation/accessories/ai-kit.html>

<https://www.raspberrypi.com/documentation/accessories/ai-kit.html>

⁴ ⁶⁰ [https://openaccess.thecvf.com/content/CVPR2023/html/](https://openaccess.thecvf.com/content/CVPR2023/html/Liu_EfficientViT_Memory_Efficient_Vision_Transformer_With_Cascaded_Group_Attention_CVPR_2023_paper.html)

[Liu_EfficientViT_Memory_Efficient_Vision_Transformer_With_Cascaded_Group_Attention_CVPR_2023_paper.html](https://openaccess.thecvf.com/content/CVPR2023/html/Liu_EfficientViT_Memory_Efficient_Vision_Transformer_With_Cascaded_Group_Attention_CVPR_2023_paper.html)

[https://openaccess.thecvf.com/content/CVPR2023/html/](https://openaccess.thecvf.com/content/CVPR2023/html/Liu_EfficientViT_Memory_Efficient_Vision_Transformer_With_Cascaded_Group_Attention_CVPR_2023_paper.html)

[Liu_EfficientViT_Memory_Efficient_Vision_Transformer_With_Cascaded_Group_Attention_CVPR_2023_paper.html](https://openaccess.thecvf.com/content/CVPR2023/html/Liu_EfficientViT_Memory_Efficient_Vision_Transformer_With_Cascaded_Group_Attention_CVPR_2023_paper.html)

⁵ ¹⁴ <https://openreview.net/forum?id=GKge9khQSa>

<https://openreview.net/forum?id=GKge9khQSa>

⁶ ⁷ ⁸ ¹³ ⁶³ ⁶⁵ [https://openaccess.thecvf.com/content/CVPR2024/papers/](https://openaccess.thecvf.com/content/CVPR2024/papers/Vasu_MobileCLIP_Fast_Image-Text_Models_through_Multi-Modal_Reinforced_Training_CVPR_2024_paper.pdf)

[Vasu_MobileCLIP_Fast_Image-Text_Models_through_Multi-Modal_Reinforced_Training_CVPR_2024_paper.pdf](https://openaccess.thecvf.com/content/CVPR2024/papers/Vasu_MobileCLIP_Fast_Image-Text_Models_through_Multi-Modal_Reinforced_Training_CVPR_2024_paper.pdf)

https://openaccess.thecvf.com/content/CVPR2024/papers/Vasu_MobileCLIP_Fast_Image-Text_Models_through_Multi-Modal_Reinforced_Training_CVPR_2024_paper.pdf

⁹ ¹⁰ ³⁰ [https://openaccess.thecvf.com/content/ICCV2023/html/](https://openaccess.thecvf.com/content/ICCV2023/html/Wu_TinyCLIP_CLIP_Distillation_via_Affinity_Mimicking_and_Weight_Inheritance_ICCV_2023_paper.html)

[Wu_TinyCLIP_CLIP_Distillation_via_Affinity_Mimicking_and_Weight_Inheritance_ICCV_2023_paper.html](https://openaccess.thecvf.com/content/ICCV2023/html/Wu_TinyCLIP_CLIP_Distillation_via_Affinity_Mimicking_and_Weight_Inheritance_ICCV_2023_paper.html)

[https://openaccess.thecvf.com/content/ICCV2023/html/](https://openaccess.thecvf.com/content/ICCV2023/html/Wu_TinyCLIP_CLIP_Distillation_via_Affinity_Mimicking_and_Weight_Inheritance_ICCV_2023_paper.html)

[Wu_TinyCLIP_CLIP_Distillation_via_Affinity_Mimicking_and_Weight_Inheritance_ICCV_2023_paper.html](https://openaccess.thecvf.com/content/ICCV2023/html/Wu_TinyCLIP_CLIP_Distillation_via_Affinity_Mimicking_and_Weight_Inheritance_ICCV_2023_paper.html)

¹¹ ¹² ⁵⁵ <https://arxiv.org/pdf/2307.12732>

<https://arxiv.org/pdf/2307.12732>

¹⁵ ⁵⁶ <https://github.com/hailo-ai/hailo-rpi5-examples>

<https://github.com/hailo-ai/hailo-rpi5-examples>

¹⁶ ³² ³³ ⁴² <https://arxiv.org/html/2504.05224v1>

<https://arxiv.org/html/2504.05224v1>

¹⁷ ¹⁸ ¹⁹ [https://openaccess.thecvf.com/content/CVPR2023/papers/Yu_X-](https://openaccess.thecvf.com/content/CVPR2023/papers/Yu_X-Pruner_eXplainable_Pruning_for_Vision_Transformers_CVPR_2023_paper.pdf)

[Pruner_eXplainable_Pruning_for_Vision_Transformers_CVPR_2023_paper.pdf](https://openaccess.thecvf.com/content/CVPR2023/papers/Yu_X-Pruner_eXplainable_Pruning_for_Vision_Transformers_CVPR_2023_paper.pdf)

[https://openaccess.thecvf.com/content/CVPR2023/papers/Yu_X-](https://openaccess.thecvf.com/content/CVPR2023/papers/Yu_X-Pruner_eXplainable_Pruning_for_Vision_Transformers_CVPR_2023_paper.pdf)

[Pruner_eXplainable_Pruning_for_Vision_Transformers_CVPR_2023_paper.pdf](https://openaccess.thecvf.com/content/CVPR2023/papers/Yu_X-Pruner_eXplainable_Pruning_for_Vision_Transformers_CVPR_2023_paper.pdf)

20 21 22 23 <https://arxiv.org/abs/2407.02068>
<https://arxiv.org/abs/2407.02068>

24 28 29 47 <https://github.com/Michel-liu/FatFormer>
<https://github.com/Michel-liu/FatFormer>

25 26 27 53 https://openaccess.thecvf.com/content/CVPR2025/papers/Wu_FIMA-Q_Post-Training_Quantization_for_Vision_Transformers_by_Fisher_Information_Matrix_CVPR_2025_paper.pdf
https://openaccess.thecvf.com/content/CVPR2025/papers/Wu_FIMA-Q_Post-Training_Quantization_for_Vision_Transformers_by_Fisher_Information_Matrix_CVPR_2025_paper.pdf

34 46 <https://www.sciencedirect.com/science/article/pii/S0031320324004758>
<https://www.sciencedirect.com/science/article/pii/S0031320324004758>

35 45 <https://pure.kaist.ac.kr/en/publications/lightweight-image-steganalysis-with-block-wise-pruning/>
<https://pure.kaist.ac.kr/en/publications/lightweight-image-steganalysis-with-block-wise-pruning/>

36 37 39 61 64 <https://www.ijcai.org/proceedings/2025/0059.pdf>
<https://www.ijcai.org/proceedings/2025/0059.pdf>

38 41 66 <https://onnxruntime.ai/docs/execution-providers/Xnnpack-ExecutionProvider.html>
<https://onnxruntime.ai/docs/execution-providers/Xnnpack-ExecutionProvider.html>

40 62 <https://onnxruntime.ai/docs/performance/model-optimizations/quantization.html>
<https://onnxruntime.ai/docs/performance/model-optimizations/quantization.html>

43 Do not overestimate RGB - Y-Scholar Hub@YONSEI
https://yscholarhub.yonsei.ac.kr/bitstream/2021.sw.yonsei/26020/2/1-s2.0-S0925231225025871-main.pdf?utm_source=chatgpt.com

44 <https://www.computer.org/csdl/journal/tp/2024/05/10330640/1SrO7sWPLJm>
<https://www.computer.org/csdl/journal/tp/2024/05/10330640/1SrO7sWPLJm>

48 <https://discuss.pytorch.org/t/fail-to-run-torch-model-on-vulkan-backend/205955>
<https://discuss.pytorch.org/t/fail-to-run-torch-model-on-vulkan-backend/205955>

49 <https://onnxruntime.ai/docs/tutorials/mobile/>
<https://onnxruntime.ai/docs/tutorials/mobile/>

50 <https://onnxruntime.ai/docs/execution-providers/community-maintained/ACL-ExecutionProvider.html>
<https://onnxruntime.ai/docs/execution-providers/community-maintained/ACL-ExecutionProvider.html>

51 52 https://www.nature.com/articles/s41598-026-42439-6_reference.pdf
https://www.nature.com/articles/s41598-026-42439-6_reference.pdf

54 58 59 <https://www.raspberrypi.com/news/raspberry-pi-ai-kit-update-dataflow-compiler-now-available/>
<https://www.raspberrypi.com/news/raspberry-pi-ai-kit-update-dataflow-compiler-now-available/>

57 <https://www.raspberrypi.com/documentation/accessories/ai-hat-plus.html>
<https://www.raspberrypi.com/documentation/accessories/ai-hat-plus.html>